Gestion des Événements et Formulaires en React

Objectifs

1. Intercepter et manipuler des événements utilisateurs via les SyntheticEvent de React.
2. Implémenter des composants contrôlés pour la saisie de données textuelles et numériques.
3. Centraliser la gestion d'un formulaire multi-champs à l'aide d'un état objet unique et du Spread Operator.
4. Mettre en place une validation synchrone en temps réel avec affichage de messages d'erreur conditionnels.
5. Gérer la soumission d'un formulaire, la prévention du comportement par défaut et le retour d'état utilisateur.

Contexte du Projet : OmniMonitor - Module d'Ajout et de Paramétrage de Stations

L'application **OmniMonitor** a besoin d'un nouveau module permettant aux opérateurs de déclarer de nouvelles stations météorologiques sur le réseau, de modifier leurs paramètres techniques et de valider les données saisies avant leur enregistrement dans le Store Redux.

Étape 1 : Gestion des Événements et Prévention du Comportement par Défaut

1. Créez un composant `src/components/ActionToolbar.jsx`.
2. Ajoutez un bouton d'action rapide qui écoute l'événement `onClick` et déclenche une fonction `handleClick(event)`.
3. Dans la console, affichez le type d'événement (`event.type`) et la cible de l'événement (`event.target`).
4. Créez un formulaire basique contenant un bouton de soumission (`type="submit"`). Dans la fonction `handleSubmit(event)`, appelez `event.preventDefault()` pour empêcher le rechargement de la page et afficher un message de confirmation dans la console.

Étape 2 : Création d'un Input Contrôlé simple

1. Créez un composant `src/components/StationSearchInput.jsx`.
2. Déclarez un état local `searchTerm` initialisé à une chaîne vide (`''`) via `useState`.
3. Créez un champ de saisie (`<input type="text" />`) dont la valeur est pilotée par l'état `value={searchTerm}`.
4. Écoutez l'événement `onChange` pour mettre à jour l'état à chaque frappe clavier (`setSearchTerm(event.target.value)`).
5. Affichez en temps réel sous l'input le texte de recherche en cours.

Étape 3 : Formulaire Multi-Champs avec un État Objet Unique

Plutôt de multiplier les variables d'état pour chaque champ, vous allez regrouper les données du formulaire dans un objet unique.

1. Créez un composant `src/components/StationForm.jsx`.
2. Déclarez un état local unique `formData` structuré ainsi :

```javascript
const [formData, setFormData] = useState({
  name: '',
  altitude: '',
  status: 'online'
});
```

3. Créez une fonction de gestionnaire de changement unique `handleChange(event)` qui extrait `name` et `value` de `event.target` et met à jour l'objet d'état en conservant les autres propriétés via le Spread Operator (`...prev`).
4. Liez chaque champ du formulaire (`input text`, `input number`, `select`) à sa clé correspondante dans l'objet via les attributs `name`, `value` et `onChange`.
5. À la soumission, interceptez l'événement et affichez l'objet complet `formData` dans la console.

Étape 4 : Validation Synchrone en Temps Réel

Vous allez ajouter une règle de validation sur le nom de la station.

1. Créez un composant `src/components/ValidatedStationForm.jsx`.
2. Ajoutez un état local pour la valeur du champ et un état dédié pour les erreurs (`const [error, setError] = useState('')`).
3. Dans la fonction `handleChange`, vérifiez la longueur du texte saisi : si le texte comporte moins de 3 caractères, définissez un message d'erreur explicite. Sinon, réinitialisez l'erreur à une chaîne vide.
4. Affichez dynamiquement le message d'erreur en rouge sous le champ de saisie si la chaîne d'erreur n'est pas vide.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- Le rechargement de la page est systématiquement bloqué lors de la soumission des formulaires via `preventDefault()`.
- Les inputs sont entièrement contrôlés par l'état local (`value` et `onChange` couplés).
- Les formulaires multi-champs utilisent un objet unique mis à jour dynamiquement grâce à l'attribut `name`.
- La validation en temps réel affiche et efface les messages d'erreur de manière fluide.